
COMPUTATIONAL ECONOMICS

Genetic Algorithms and Portfolio Models in MATLAB

David A. Kendrick
Ruben P. Mercado
Hans M. Amman

1 Introduction

In this chapter we present an example that builds on the Markowitz optimal portfolio model that we dealt with in the Lecture on *Portfolio Modeling in Matlab*. In that chapter we used two different methods to solve the Markowitz problem: first a Monte Carlo optimization search method and then a **MATLAB** gradient optimization function. Here we use a genetic algorithm based on the one we discussed in the Lecture on *Portfolio Modeling using a Genetic Algorithm*

First we solve the convex problem from the Lecture on *Portfolio Modeling*. It has a unique global maximum—given the quadratic nature of the criterion function to be optimized. Later in this chapter we introduce a more difficult but more realistic problem by including brokerage fees, which may result in nonconvexities and thus in a number of local maxima. It is for this kind of problem that genetic algorithms are particularly useful, as they are global optimization algorithms. They perform a global exploration of the optimization space and are less likely to be trapped by local minima or maxima than is the case for other standard optimization procedures.

2 Overview of the MATLAB Code

Remember that the Markowitz problem was stated earlier in terms of finding x to maximize J in

$$J = \mu'x - \frac{1}{2}\beta x'\Sigma x \quad (1)$$

subject to the constraints

$$\sum_{i \in I} x_i = 1 \quad (2)$$

$$x_i \geq 0 \quad i \in I \quad (3)$$

where

J = criterion value

β = subjective weight on the variance of the return on the portfolio

x_i = the fraction of the portfolio invested in equity i

I = the set of equities

$$x = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}$$

$$\mu = \begin{bmatrix} \mu_1 \\ \mu_2 \\ \mu_3 \end{bmatrix} = \begin{bmatrix} 8 \\ 12 \\ 15 \end{bmatrix}$$

$$\Sigma = \begin{bmatrix} \sigma_{11} & \sigma_{12} & \sigma_{13} \\ \sigma_{21} & \sigma_{22} & \sigma_{23} \\ \sigma_{31} & \sigma_{32} & \sigma_{33} \end{bmatrix} = \begin{bmatrix} 6 & -5 & 4 \\ -5 & 17 & -11 \\ 4 & -11 & 24 \end{bmatrix}$$

The main program we use here to solve the Markowitz problem, which is a genetic algorithm portfolio problem, is `gaportfol1.m`. This program and all the functions it calls are available in the book web page.

The basic structure of the program, which follows, is analogous to the program `gagame.m` presented in Chapter 11. It consists of three main parts: the first part contains the initialization of counters and parameters and a function call to initialize the population; the second is a for loop across generations that in turn contains several function calls; and the third contains commands to print and graph the main results:

```
% Genetic Algorithm portfolio program
% with initpopdet and parentsdet
% Program name: gaportfol1.m
% On an early version by Huber Salas

% initialization of counters and parameters;
nruns = 100;    popsize = 8;

beta = 2;
mu = [8 12 15]';
sigma = [6 -5 4;
        -5 17 -11;
        4 -11 24];

num = 3;    clen = num * 8;    pmut = 0.5;

% generation of chromosome strings of initial population
genepool= initpopdet(popsize);
```

```

for k = 1:nruns;

% transformation of crhromosome string into normalized n-asset portfolio
pwm = normport(genepool,popsize,clen,num);

% computation of fitness function and fittest individual
[fit, bestind, bestfit] = fitness_gaportfol(pwm,mu,popsize,beta,sigma);
wbest(:,k) = bestind;
    fbest(k) = bestfit;

% selection of parents;
[parent0,parent1] = parentsdet(fit,genepool);

% crossover of parents chromosome strings
[child0,child1] = crossover(clen,parent0,parent1);

% mutation of children chromosome strings
for h = 1:2:popsize;
    child0mut = mutation(pmut,clen,child0);
    genenew(h) = child0mut;
    child1mut = mutation(pmut,clen,child1);
    genenew(h+1) = child1mut;
end
    genepool= genenew;
end

% print and graph optimal weights and criterion;
wbest
fbest
figure(1);
xaxis = [1:1:nruns]';
plot(xaxis,wbest(:,,:));
figure(2);
xaxis = [1:1:nruns]';
plot(xaxis,fbest(:,,:));

```

In the initialization of counters and parameters section we set the number of runs `nruns`, the population size `popsize`, the parameters of the portfolio model (the risk aversion coefficient `beta`, the vector of mean returns `mu`, and the variance-covariance matrix `sigma`), and the number of assets `num`. We also set the length

of the chromosome string `clen` to be used to represent each portfolio as equal to the number of assets `num` times eight, that is, twenty-four bits. Finally, we set the probability of a child mutation `pmut`.

We then call the function `inipopdet` to initialize the vector `genepool`, which contains a number of portfolios equal to the population size, each portfolio represented by a twenty-four-bit chromosome string. Thus, in our example, `genepool` is a vector of twenty-four-bit chromosomes with an element for each of the eight individuals in the population.

Then we move on to the main for loop in the program, running from 1 to the number of runs. This loop contains a sequence of function calls. It starts with a call to the function `normport` to transform each twenty-four-bit chromosome string corresponding to a portfolio into an equivalent normalized weight three-asset portfolio. The vector `genepool` may be thought of either as a $(1 \times \text{popsize})$ vector of integers or as a $(1 \times \text{popsize})$ vector of twenty-four-bit strings. Thus the `normport` function transforms the $(1 \times \text{popsize})$ vector `genepool` into the $(3 \times \text{popsize})$ portfolio weight matrix `pwm`, which contains the normalized weights of each three-asset portfolio in each column.

Next follows a call to the `fitness_gaportfol` function to compute the fitness function for each individual (each portfolio) and to select the fittest individual, which at each run is stored in the `kth` column of the matrix `wbest` while the corresponding criterion value is stored in the `kth` element of the vector `fbest`. Thus, at the end of the runs, this matrix and vector contain the sequence of optimal portfolios and optimal criterion values, respectively.

Next the function `parentsdet`, using the fitness function previously computed, selects two parents (`parent0` and `parent1`), who form a couple. This is followed by a call to the function `crossover`, which generates two children (`child0` and `child1`) as the product of the crossover of the chromosome strings of the two parents.

Next comes a for loop whose index goes from 1 to `popsize` in increments of two. In this loop, a new generation is created out of the two newborn children through mutations of their chromosomes. Half of the new generation comes out of mutations of the first child (`child0`) and the other half out of mutations of the second child (`child1`). At every pass through the loop the function `mutation` is called, which generates a mutated child, and its twenty-four-bit chromosome representation is stored in a cell of the `genenew` vector. Once the new generation is created, the new vector `genenew` replaces the old vector `genepool` and the main loop of the program

starts over again.

Finally, once the main loop goes through the established number of runs, the matrix of fittest individuals `wbest` and the vector of optimal criterion values `fbest` are printed and plotted.

This provides an overview of the program. It is important to point out that every time you run the program, particularly when changing the number of generations or the population size, you should clean out the old commands and workspace to avoid displaying spurious results. To do so, go to Edit in the top **MATLAB** menu. Then select Clear Command Window and confirm with Yes that you want to do this. Then do the same for Clear Command History and for Clear Workspace.

We now present each function in detail.

3 Functions

3.1 Initpopdet

The `initpopdet` function is simple in that all it does is assign the same portfolio weights (33%) to each asset in each portfolio as in the experiments performed in Chapter 7, so that we can easily compare results. However, the function is made more complicated by the necessity to represent a row vector of the three weights, that is,

[33 33 33]

with a single twenty-four-bit string such that each eight-bit segment of the string represents the number 33, that is, 00100001. Thus the twenty-four-bit string is

00100001 00100001 00100001

This string has 1's in the positions 0, 5, 8, 13, 16 and 21 (counting from the right to left beginning with 0), so its integer value is

$$(1) 2^{21} + (1) 2^{16} + (1) 2^{13} + (1) 2^8 + (1) 2^5 + (1) 2^0$$

$$w = 2^{21} + 2^{16} + 2^{13} + 2^8 + 2^5 + 2^0;$$

The **MATLAB** for this initialization function is thus

```
function genepool= initpopdet(popsiz);  
w = 2^21+2^16+2^13+2^8+2^5+2^0;
```

```
dec2bin(w,24)
genepool= w * ones(1,popsize);
```

The header statement for the function,

```
function genepool = initpopdet(popsize);
```

tells us that the name of the function is `initpopdet`, that the argument `popsize` is passed to the function, and the result `genepool` is returned by the function. After `w` is defined in the first statement in the function, the statement

```
dec2bin(w,24)
```

appears. It does not play an essential role in the function since it only serves to print a binary representation of `w` for debugging purposes. Since there is no semicolon at the end, this statement returns and prints the twenty-four-bit binary representation of `w`. Finally, a $(1 \times \text{popsize})$ vector of 1's is multiplied by the previously created cell `w` to obtain the initial population vector `genepool`. Thus, in the case at hand `genepool` is a vector of eight twenty-four-bit strings. Finally, note that an end statement is not necessary.

We will see later, in the `initpoprand_gaportfol` function, how to replace this rudimentary function by a more sophisticated random procedure to initialize the population.

3.2 Normport

The `normport` function takes the twenty-four-bit chromosome string corresponding to each portfolio and creates an equivalent normalized three-asset portfolio representation. Thus, it transforms the $(1 \times \text{popsize})$ vector `genepool` into the $(3 \times \text{popsize})$ portfolio weight matrix `pwm`, which contains the normalized weights (i.e., the weights add to 1) in each column of each three-asset portfolio. The complete function is follows:

```
function pwm = normport(genepool,popsize,clen,num);
genetemp = genepool;
for i = 1:popsize;
n=ceil(clen/num);
mask = 2^n-1;
port = zeros(1,num);
    for j = 1:num
        port(num-j+1) = bitand(genetemp(i),mask);
```

```

        genetemp(i)=bitshift(genetemp(i),-n);
    end
    port = port/sum(port);
    pwm(:,i) = port';
end

```

Since we are using a chromosome with twenty-four bits and have three assets in the portfolio, we have eight bits to specify the proportion of the portfolio held in each of the assets. Since eight bits permits us to specify integers from 0 to 255 we have an accuracy of about 0.5 percent in the solution to the portfolio problem. So the first step after creating the temporary variable `genetemp` is to determine the number of bits, `n`, used for each equity by dividing the chromosome length by the number of equities. This is done with the statement

```
n = ceil(clen/num);
```

where `ceil` is a **MATLAB** function that returns an integer that is the ceiling, that is, the round off of a decimal to the nearest integer greater than the number. Also recall that `clen` is the chromosome length, twenty-four, and `num` is the number of assets in the portfolio, three, so in our case this becomes `ceil(24/3)` or eight. This statement ensures that there is an integer number of bits to represent the percentage of each stock held in the portfolio.

The next step is to pull out the `n` bits in the chromosome that correspond to the proportion for each equity. This is done by creating a mask in which the lower-order `n` elements are 1's and all other bits are 0. This is done with the statement

```
mask = 2^{\}n-1;
```

Keeping in mind the precedence rules, we know that this is $2^n - 1$. Thus in our case with `n` equal to eight, `mask` is an integer variable with value 255 and its binary representation is the twenty-four-bit string

```
00000000 00000000 11111111
```

and we can use a bitand operation with this mask to pick off eight-bit sections of the chromosome.

The next step is to initialize the vector that carries the percentage allocation of elements in the portfolio. In our case this is a three-element vector of integers that is initialized with the statement


```
port = zeros(1,num);
```

This vector is then used in a for j loop over the number of equities in the portfolio to get the bit string for each element in the portfolio as follows:

```
for j=1:num
port(num-j+1) = bitand(genetemp(i),mask);
genetemp(i) = bitshift(genetemp(i),-n);
end
```

The mask variable is used on each pass through the for loop to put the lower-order eight-bit section of the chromosome into the variable port. In addition, in each pass the chromosome is shifted to the right by eight bits and filled with 0's in the leftmost eight bits using the bitshift operation.

Thus if we begin with a chromosome like

```
00000110 11000000 11100111
```

the first pass through the loop would put the bit string

```
00000000 00000000 11100111
```

into the variable port(3) and the second pass would put the bit string

```
00000000 00000000 11000000
```

into the variable port(2), and so on.

Since the port variables are now integers with values between 0 and 255 they must be normalized by the sum of their values to convert them to percentages of the portfolio. This is done with statement

```
port = port/sum(port);\
```

Finally, the transpose of the three-element vector port is stored in the corresponding column of the matrix pwm with the statement

```
pwm(:,i) = port';
```

3.3 Fitness_gaportfol

This fitness_gaportfol function uses a procedure similar to the one in the Lecture on *Portfolio Modeling* using **Matlab**

```

function [fit,bestind,bestfit] = fitness_gaportfol(pwm,mu,popsize,beta,sigma);
pret = pwm' * mu;
for j = 1:popsize;
    pvar(j) = 0.5 * beta * pwm(:,j)' * sigma * pwm(:,j);
end
fit = pret - pvar';
[top topi] = max(fit);
bestind = pwm(:,topi);
bestfit = top;

```

It begins with a statement to compute the vector of returns `pret` as the product of the portfolios times the corresponding returns. Then statements are included to compute the vector of portfolio variance costs `pvar` and the corresponding criterion function vector, or fitness vector `fit`. In our case the vector `fit` is an eight-element vector that provides the fitness level for each of the individuals in the population.

The statement

```
[top topi] = max(fit);
```

then returns the value (`top`) and the index (`topi`) corresponding to the maximum value in the fitness vector `fit`. Finally, the `topi` index is used to assign to the *best individual* vector, `bestind`, the corresponding normalized three-asset vector with the statement

```
bestind = pwm(:,topi);
```

while the corresponding value of the criterion function vector is assigned to the variable `bestfit` with the statement

```
bestfit = top;
```

With the fitness now determined we turn next to the selection of parents, crossover, and mutation.

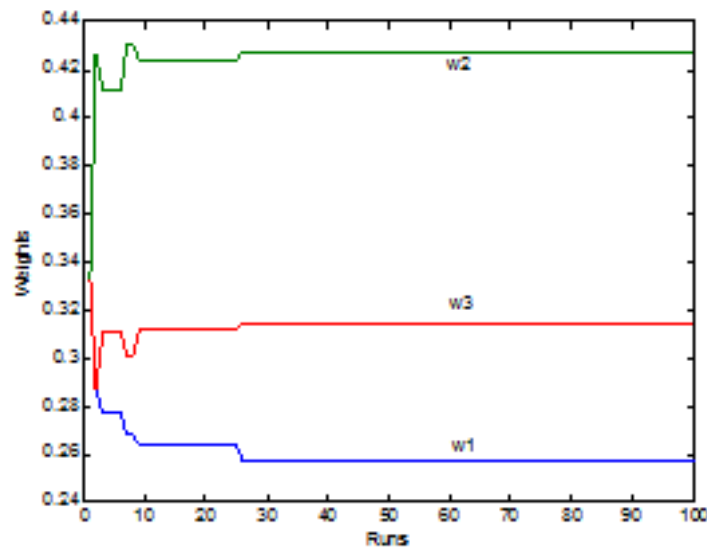
3.4 Parentsdet, Crossover, and Mutation

The `parentsdet`, `crossover`, and `mutation` functions are exactly the same as the ones used in Chapter 11. They even have the same variable names. Thus they can be used by the program in that chapter as well as the programs in this one.

4 Results

Figure 12.1 shows the results of running the main program `gaportfol.m` with a number of runs equal to 100 and a population size of eight. The optimal values of the portfolio weights for the last run are $w_1 = 0.26$, $w_2 = 0.42$, and $w_3 = 0.32$, which is slightly different than the results we obtained in the Lecture on *Portfolio Modeling*. We can see how, after starting from initial values equal to 0.33, the weights converge to the optimal values. We can also see how the criterion value converges to a value of 9.44.

Figure 1: Runs

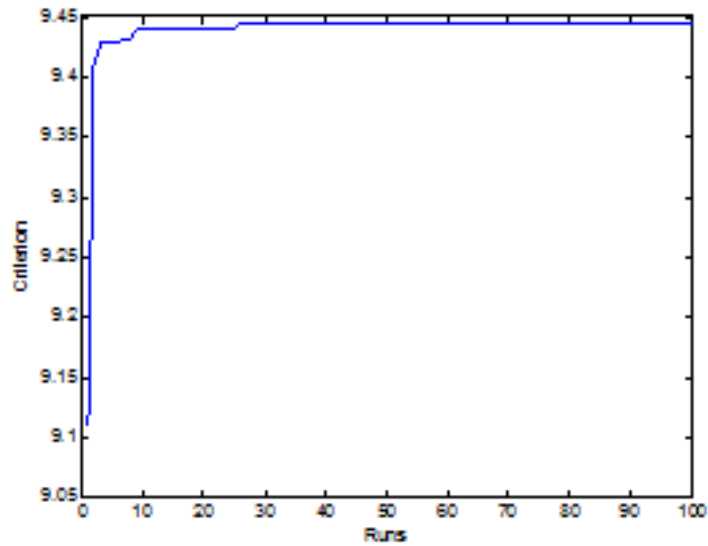


<Figure 12.1>

5 Refinements

The program `gaportfol1.m` and its functions give us a basic idea of the work with genetic algorithms. However, some of its functions are quite rudimentary and they do not take us beyond what we learned in Chapter 11. In this section we introduce some alternative and more sophisticated functions and main program structure. They can also be used to develop a more sophisticated version of the `gagame.m` program presented in Lecture on *Genetic Algorithms* using Matlab.

Figure 2: Genetic algorithm portfolio example



5.1 Initpoprand_gaportfol

With the `initpopdet` function we generated, in a deterministic way, an initial vector of portfolios all with the same weights. However, it is customary in the field of genetic algorithms to generate the initial population randomly. To do so, we introduce the `initpoprand_gaportfol` function. In Chapter 11 we also generated the initial population randomly—in the `initpoprand_gagame` function—but with a different procedure. Here we present an alternative:

```
function genepool= initpoprand_gaportfol(clen,popsize);
mask = 1;
genepool= zeros(1,popsize);
for j = 1:popsize
    for i = 1:clen
        if (rand < 0.5)
            genepool(j) = bitor(genepool(j),mask);
        end
        genepool(j) = bitshift(genepool(j),1);
    end
    genepool(j) = bitshift(genepool(j),-1);
end
```

The key elements of this function are a set of two nested for loops

and an if statement. The index of the for loop over individuals in the population is j and the index over the bits in the binary representation of the chromosome for each individual is i . Also the i loop runs from 1 to $clen$, which is the number of binary elements in the chromosome—in the present case twenty-four.

Moreover, each bit in the chromosome for individual j is modified, that is, switched from zero to one with probability $1/2$ since `rand` is a zero-one uniform distribution random number generator.

Next focus for the moment only on the portion inside the for loop for the number of bits. The key element in this segment of code is the bitor operation. We know from the foregoing that `genepool(j)` has been initialized to zero and the variable `mask` has been initialized to one; therefore the bitor operation applied to these two strings yields

```
00000000 00000000 00000001
```

on the first pass through the i loop if the call to `rand` yields a value that is less than 0.5. We also know that this occurs half the time.

Following the if statement, we use the operation `bitshift` to shift the binary string `genepool` one position to the left and to put a zero in the rightmost (lowest-order) position. After the `bitshift` operation the foregoing string becomes

```
00000000 00000000 00000010
```

Thus in the i loop the bits in the `genepool(j)` string are considered one by one and changed from 0 to 1 with probability 0.5. Finally, one can see in the foregoing code that `bitshift` is used to move the bit string one step back to the right after the end of the i loop. Otherwise the last bit in the string will always be equal to zero.

To use this function, we have to substitute in the program `gaportfol.m` the following statement for the call to the function `initpopdet` in the *generation of chromosome string of initial population* section

```
genepool = initpoprand_gaportfol(clen,popsize);
```

Figure 4 shows the results of running program `gaportfol2.m`, which is the `gaportfol1.m` program with the `bitshift` function. The optimal

Figure 3: Runs

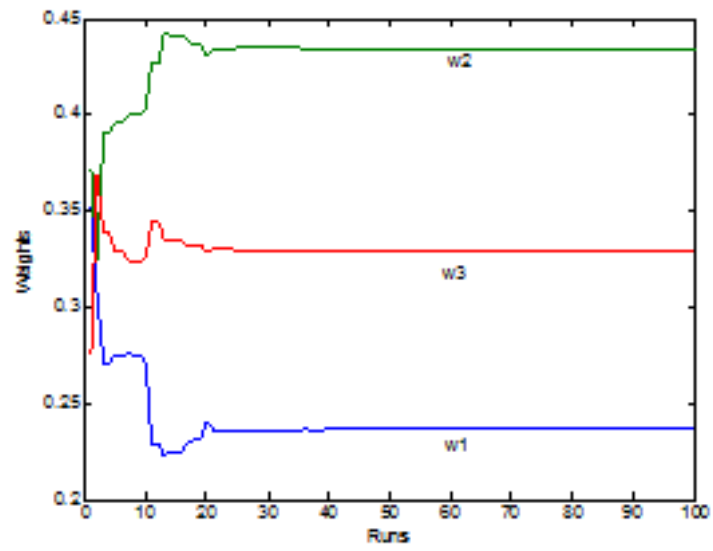
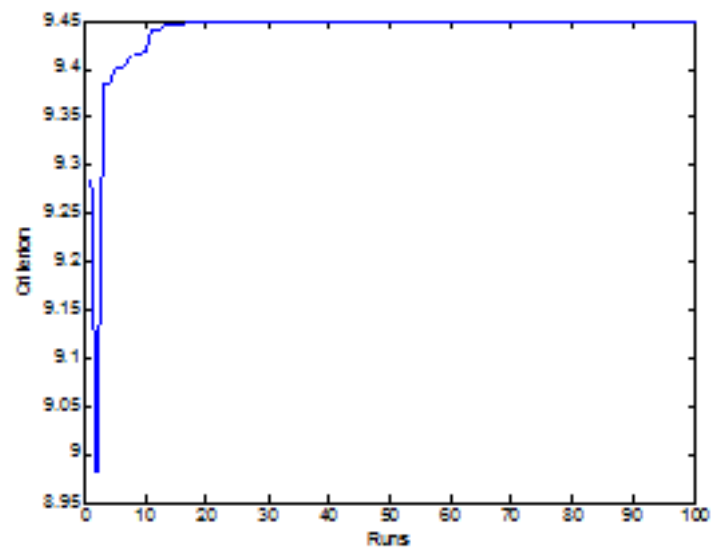


Figure 4: Runs



portfolio weights are here $w_1 = 0.24$, $w_2 = 0.43$, and $w_3 = 0.33$, which are slightly different than the results obtained in Chapter 7. Moreover, as expected, the path of the optimal weights starts from random locations instead of from the 0.33 value as in Figure 3. The criterion value, after decreasing during the initial runs, converges to a value of 9.45.

5.2 Parentsrand

In programs `gaportfol1.m` and `gaportfol2.m` we used the function `parentsdet` to select the two parents of a new generation. That was a deterministic procedure in which we picked as parents the two individuals with the highest and next to the highest value of the fitness function. However, the usual practice in the genetic algorithms field is to introduce some randomness in the selection of parents. The function `parentsrand` (parents random) we present in this section is a first step in that direction.

The method used can be thought of as a cross between a pie chart and a roulette wheel. Consider a case in which there are five individuals in the population and all five have the same fitness level of 40. We could then use a pie chart to represent the percentage of the total fitness of the population of 200 that is held by each individual as shown in Figure 5. One could also think of this pie chart as a roulette wheel that is spun each time a mate is to be selected. Since all the slices of the pie are the same size, the probability of an individual being selected as a mate would be the same for all of them.

However, consider instead a case in which the fitness of each of the five individuals is different, say 20, 60, 30, 20, and 70. Then the pie chart representing the percentage of the total fitness held by each of the individuals would look like Figure 6. In this case each time the roulette wheel is spun there would be decidedly different probabilities that any particular individual would be chosen. The function code is as follows:

```
function [parent0,parent1] = parentsrand(fit,genepool);
[top topi] = max(fit);
parent0 = genepool(topi);
fit(topi) = 0;
[top topi] = max(fit);
```

Figure 5: A balanced pie chart/roulette wheel

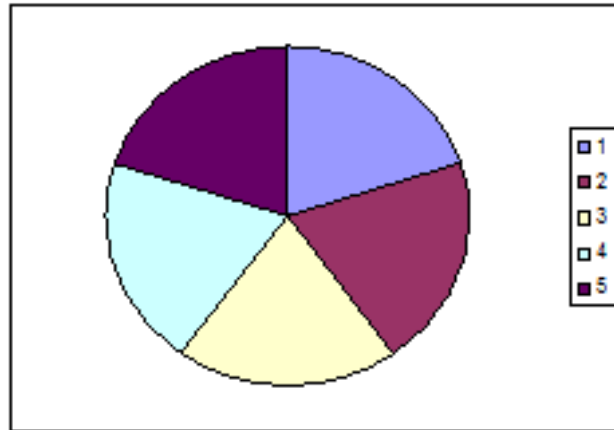
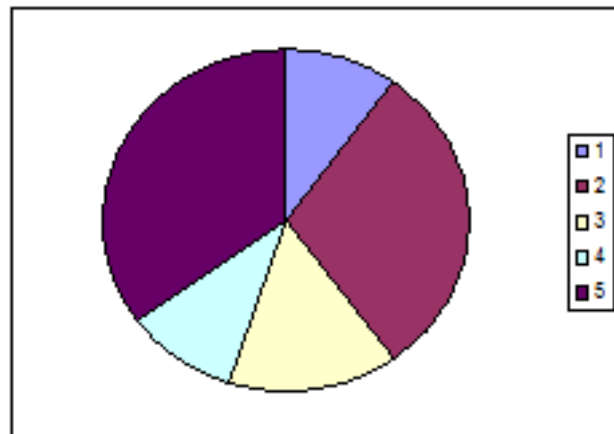


Figure 6: An unbalanced pie chart/roulette wheel




```
parent1 = genepool(topi);
```

The variable `cumfit` is the cumulative value of the individual fitness variables, that is, the sum of the fitness levels of the members of the population. The variable `val` is used to move around the roulette wheel as it is spun and the variable `spin_val` carries the information about how far the wheel travels before it stops. The variable `rand` provides a number from the zero-to-one interval of a uniform distribution. Thus each time the roulette function is called, `spin_val` takes on a different value, which ranges uniformly from zero to the sum of the fitness levels. However, the compartments of the roulette wheel are not all the same size; rather they represent the relative fitness of the individuals, so more-fit individuals are more likely to be chosen.

The selection occurs in the while loop, which repeats until the variable `val` exceeds `spin_val` or until the loop index `j` exceeds the population size. Moreover, each time through the loop the variable `val` is augmented by the fitness level of individual `j`. After the completion of the while loop, the index of the selected individual is then used to choose the corresponding chromosome string from the vector `gen`, and this information is transferred to the variable `f`, which is the output of the function.

One thing to note about this procedure is that it is *sampling with replacement*, that is, when a mate is chosen that individual is not removed from the population but rather is left there and is eligible to be chosen on subsequent calls to the function.

To use this function, we have to replace in the program `gaportfolio.m` the call to the function `parentsdet` with the call to the function `parentsrand` in the *selection of parents* section in the following two statements:

```
parent0 = parentsrand(fit,popsize,genepool);  
parent1 = parentsrand(fit,popsize,genepool);
```

Figure 7 shows the results of running program `gaportfolio3.m`, which is program `gaportfolio1.m` with this function and using the `initpoprand_gaportfolio` function as explained in the previous section. We see here that the patterns of optimal portfolio weights and criterion values, at variance with Figures 3 and 4, are very unstable. Why? The reason may be that here we are choosing the two parents with some randomness, whereas in the other two cases we always chose the two best performing individuals to

Figure 7: Runs

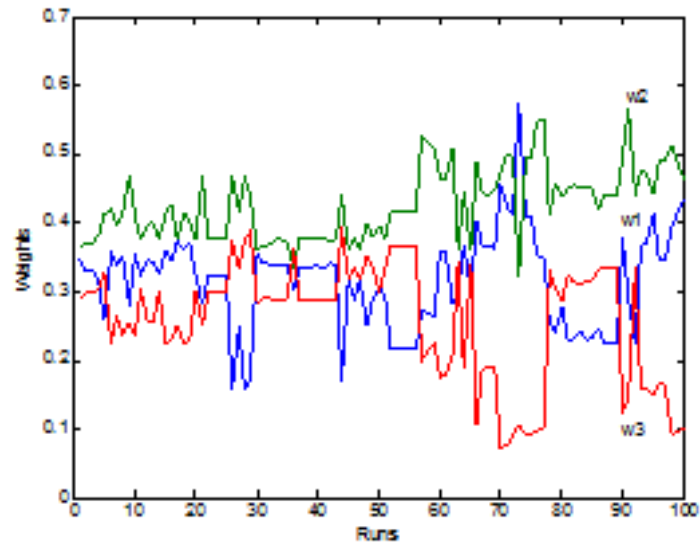
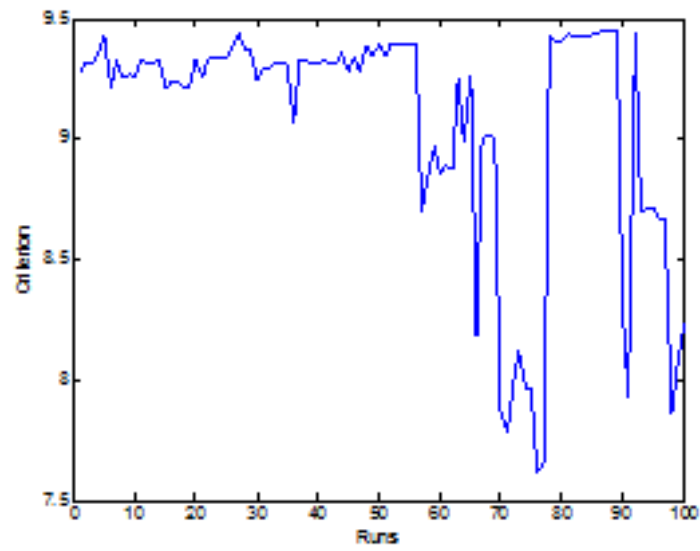


Figure 8: Runs



form a couple. This result is important to make the point that genetic algorithms are designed to perform a wide exploration of the solution space. Thus, they usually work with large populations and a large number of runs. If we increase those values we observe some performance improvement, that is, with 100 runs as before but now with a population of 500 instead of eight. However, before doing so it is convenient to adjust the program `gaportfol3.m` to widen the pool of best couples as we will see in the next section.

5.3 Selecting More Than One *Best Couple*

In the program `gaportfol3.m` we introduced randomness in the initialization of the population and in the selection of parents. However, we followed the same procedure we used in the previous programs for parent selection, that is, we just selected the couple with the highest criterion value to give birth to the entire new generation. However, to make a better use of the random selection of parents process introduced earlier, that is, to widen the search space of an optimum, thus reducing the chances of being trapped in a local optimum, it may be convenient to obtain the new generation of children from more than just one couple. This can be accomplished, for example, by extending the range of the children's generation loop in the *mutation of children chromosome strings* section from the `gaportfol3.m` program,

```
% mutation of children chromosome strings
for h = 1:2:popsiz;
    child0mut = mutation(pmut,clen,child0);
    genenew(h) = child0mut;
    child1mut = mutation(pmut,clen,child1);
    genenew(h+1) = child1mut;
end
```

to make it cover the two previous sections as well, namely to include the calls to the `parentsrand` and `crossover` functions:

```
for h = 1:2:popsiz;
% selection of parents;
parent0 = parentsrand(fit,popsiz,genepool);
parent1 = parentsrand(fit,popsiz,genepool);

% crossover of parents' chromosome strings
```

```

[child0,child1] = crossover(clen,parent0,parent1);

% mutation of children chromosome strings

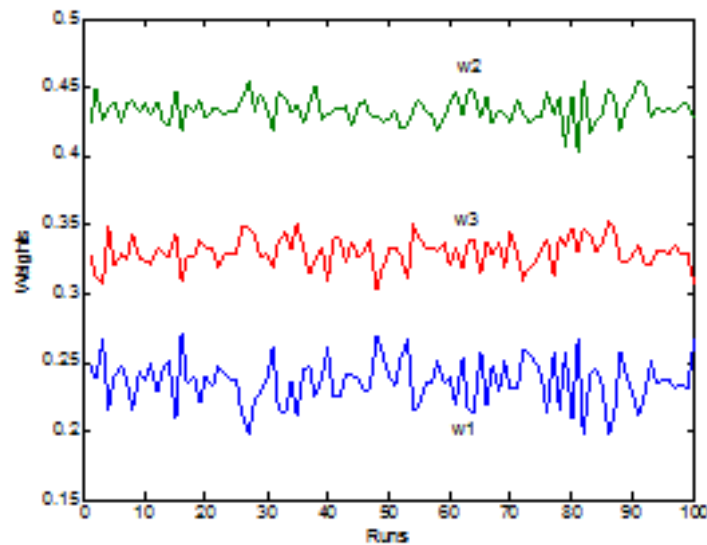
child0mut = mutation(pmut,clen,child0);
genenew(h) = child0mut;
child1mut = mutation(pmut,clen,child1);
genenew(h+1) = child1mut;

end

```

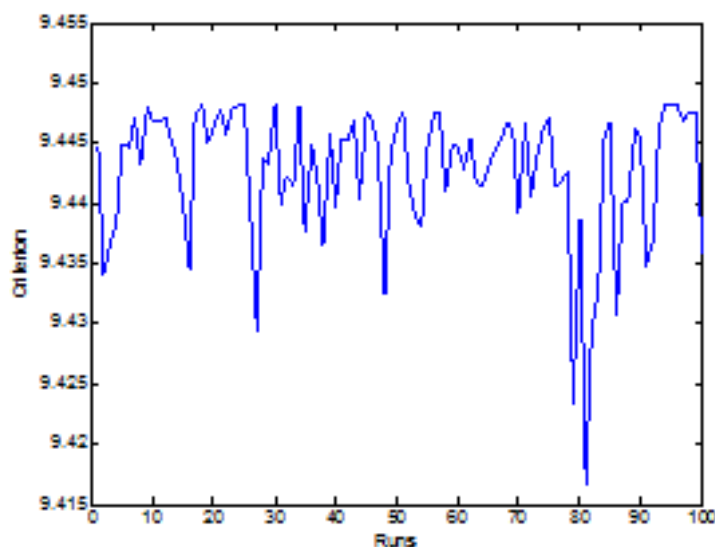
With this change, at each pass of the loop, each *best* couple randomly selected in the *selection of parents* section gives birth to only two children, who in time experience mutations. Figure 8 shows the result of an experiment with the program `gaportfol4.m`, which incorporates this change, for 100 runs with a population of 500.

Figure 9: Runs



We see that the performance improves in the sense that the weights and the criterion values follow a more discernible pattern, but it is still more unstable than in Figures 3 and 4. However, as we said earlier, the risk of being trapped in a local optimum in the case that a number of them exist is expected to be lower.

Figure 10: Runs



6 A More Difficult Portfolio Problem

So far we have been working with a Markowitz-type portfolio optimization problem with a quadratic criterion function. This is by nature a convex problem with a single optimum. However, genetic algorithms are usually employed to solve more complex problems that may have a number of local optima and be difficult for local optimization methods, such as gradient or Newton methods, to solve, but easier for global optimization methods such as genetic algorithms.¹

In the previous convex problem the criterion (fitness) value was the mean return (revenue) minus the variance cost, ignoring the brokerage cost for purchasing equities. In this problem we change that to the *profit* minus the variance cost, where the profit is the mean return less the brokerage fee. Moreover, we use a realistic form of the brokerage fee that includes both a fixed and a marginal cost for the purchase of each type of equity. This has the effect of making the average cost of purchasing equities decline with the number of equities purchases, which in turn raises the possibility that the optimization problem may have local optima. We have also imposed the restriction that the individual must purchase a percentage above some lower bound of each type of equity.

¹For some other approaches to global optimization see Goffe (1996) and Tucci (2002).

Thus if the selection of parents, crossovers, and mutations generates a portfolio in which one or more equities are below the lower bound, this amount is reset to zero and the small amount is redistributed to the other stocks.

The code itself for this second problem is similar to that for the problem in the previous section, with the exception that the `fitness_gaportfol` function is now replaced with the `fitnessnc` function. This function has two main parts. The first one, dealing with the portfolio redistribution, follows:

```
function [fit,bestind,bestfit] = fitnessnc(pwm,mu,popsize,beta,sigma,num);

% Portfolio Redistribution
for j = 1:popsize;
    cond = ones(num,popsize);
    counter = num;
    acum = 0;

    for i = 1:num;
        if pwm(i,j) < 0.1
            acum = acum + pwm(i,j);
            pwm(i,j) = 0;
            cond(i,j) = 0;
            counter = counter - 1;
        end
    end

    for i = 1:num;
        if counter > 0
            if cond(i,j) == 1;
                pwm(i,j) = pwm(i,j) + (acum / counter);
            end
        end
    end
end
```

There is a main for loop, running across portfolios from 1 to `popsize`—the population size—and two inside loops running across each portfolio from 1 to `num`—the number of equities. The main loop begins by

defining three auxiliary variables that are reset at each pass of the loop through each portfolio. The variable `cond` is defined as comprised of 1's and with the same dimensions as the variable `pwm`. It is initialized with 1's and is used to mark with 1's those equities whose amount is above the allowed lower bound and with 0's otherwise. The variable `counter` is used to count the number of equities in each portfolio whose amount is above the lower bound. The variable `acum` contains the accumulated amount of stocks below the lower bound in each portfolio.

The first inside loop follows. When the amount of an equity in the portfolio is below the lower bound—set to 0.1 in the present example—that amount is accumulated to be redistributed later. Then, that equity's participation in the portfolio is set to zero and marked with a 0 in its corresponding location in the `cond` matrix. Finally, the counter of the number of equities above the lower bound in the portfolio is decreased by one unit.

The second inside loop begins by checking that the variable `counter` is different from 0 to avoid a possible division by 0 later. Then a conditional statement checks if the corresponding equity is marked with a 1, thus being above the lower bound. In this case, the corresponding proportional amount of previously accumulated stocks to be redistributed is added to that equity.

The second main part of the function, dealing with the computation of portfolio returns and the selection of the best portfolio is shown in the following, which is very similar to the `fitness_gaportfolio` function corresponding to the convex example, with some minor differences:

```
% Computation of portfolio returns and best portfolio

fc = 0.2 * ones(1,num);
mc = 0.05 * ones(1,num);
pret = pwm' * mu;
for j = 1:popsiz;
    pvar(j) = 0.5 * beta * pwm(:,j)' * sigma * pwm(:,j);
end

for j = 1:popsiz;
    pbrok(j) = fc * cond(:,j) + mc * pwm(:,j);
end
```

```

fit = pret - pvar' - pbrok';
[top topi] = max(fit);
bestind = pwm(:,topi);
bestfit = top;

```

The first difference is that it is necessary to define vectors of fixed and marginal cost terms for the brokerages fees. This is done in the initialization section of the function with the statements

```

fc = 0.2 * ones(1,num);
mc = 0.05 * ones(1,num);

```

Thus in these vectors we allow for different fixed costs and marginal costs for the various types of equities. However, we have treated these costs as the same for across equities in the present example. We have purposely made the fixed cost relatively large in order to increase the likelihood that the problem will have local optima. Then we compute the brokerage cost for each portfolio with the loop

```

for j = 1:popsiz;
    pbrok(j) = fc * cond(:,j) + mc * pwm(:,j);
end

```

where pwm is the matrix containing each vector of portfolio weights and cond is the matrix containing each vector of portfolio marks, with 1's for equities above the lower bound and 0's otherwise. Thus, the fixed cost is charged on portfolios above the lower bound only. Finally, as in the `fitness_gaportfol` function, we compute the fitness of each individual (now including the brokerage cost) and select the fittest one.

To solve this nonconvex example we use a modified version of the program `gaportfol4.m`, which we name `gaportfol5.m`. In the *computation of fitness function and fittest individual* section of `gaportfol4.m`, we have to replace the previous `fitness_gaportfol` function call with the statement

```

fit, bestind, bestfit] = fitnessnc(pwm,mu,popsiz,beta,sigma,num);

```

Figure 12 shows the result of running the program `gaportfol5.m`.

The results are similar to those shown in Figure 11. However, they may change significantly if we introduce substantial changes in the fixed costs and/or in the equities lower bound.

Figure 11: Runs

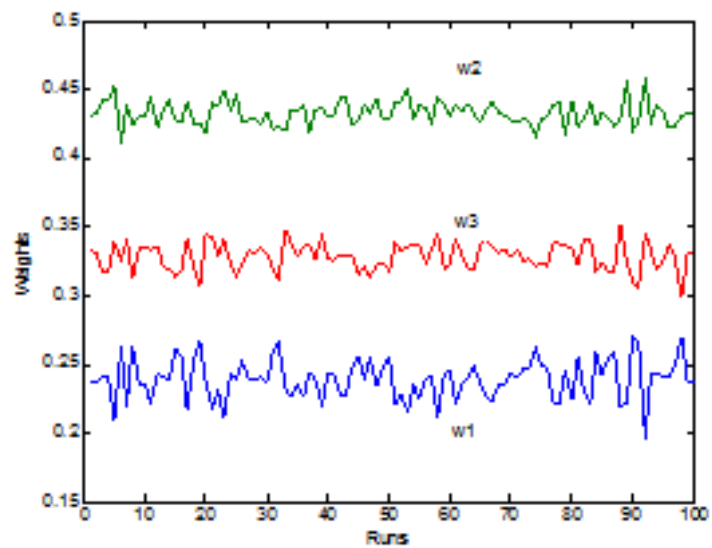
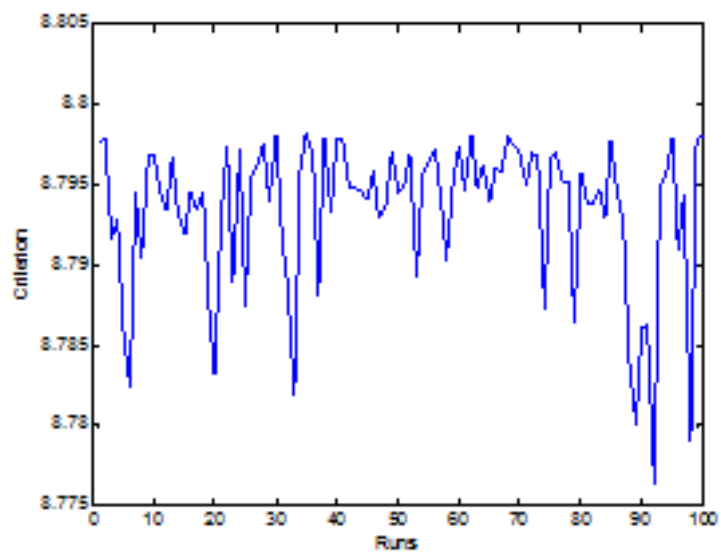


Figure 12: Runs



7 Experiments

Just as in the Lecture on *Portfolio Modeling*, the simplest experiments with this genetic algorithm code involve changing the means and/or variances of the portfolio and/or the brokerage costs to see how the weights of the best portfolios change in response. Another simple experiment would be to change the number of model iterations and/or population size to see how this affects the outcome.

A more interesting set of experiments is to introduce further refinements in the code to move closer to actual practice in the field of genetic algorithms. A first refinement would be to introduce mutations in more than one bit in the children's chromosomes in the mutation function. A second would be to introduce more than a single crossover point in the crossover function.

8 Further Reading

A classic reference in the genetic algorithm literature is Goldberg (1989). For financial applications, see Bauer (1994).

References

- Bauer, R. J.: 1994, *Genetic Algorithms and Investment Strategies*, John Wiley, New York, New York.
- Goldberg, D.: 1989, *Genetic Algorithms in Search, Optimization and Machine Learning*, Addison-Wesley, Cambridge, Massachusetts.